

Tema 5

Ingeniería de Características con IA Generativa

PSU — IA Generativa Aplicada al Análisis de Datos

Universidad Internacional de La Rioja (UNIR)

Mayo 2026

1. ¿Qué es el feature engineering y para qué sirve?

La ingeniería de características (feature engineering) es el proceso de transformar datos crudos en representaciones que maximicen la señal informativa disponible para un modelo de ML.

Principio central (Domingos, 2012)

El éxito en ML no depende únicamente del algoritmo, sino de cómo se representan los datos. Dos modelos idénticos pueden tener rendimientos drásticamente distintos si trabajan con conjuntos de características diferentes. Un modelo sencillo con buenas variables puede superar a un modelo complejo con malas variables.

Un modelo de ML solo puede aprender patrones a partir de lo que recibe como entrada. Si la información predictiva no está *explícitamente representada* en las variables, el modelo no puede capturarla — o la captura con mucha dificultad a través de múltiples splits sucesivos.

Ejemplo motivador — Churn en telecomunicaciones

Dataset con variables: `consumo_gb` , `llamadas_soporte` , `antiguedad_meses` . La hipótesis de negocio es que los clientes con *bajo consumo y alta interacción con soporte* son los que se van. Un Random Forest podría capturar esa interacción a través de múltiples splits. Pero si se crea directamente una variable `friccion_reciente = 1` si `consumo_gb < 12 AND llamadas_soporte >= 3` , se le entrega la señal masticada al modelo. Eso es feature engineering.

1.1 Criterios para crear una variable nueva

No toda variable nueva es útil. Una variable nueva debe cumplir tres criterios para justificar su inclusión:

Criterios de validez de una nueva característica

1. Aporte de información nueva:

no debe ser redundante con variables existentes. La colinealidad — cuando la variable nueva puede predecirse casi perfectamente a partir de las existentes — anula su contribución marginal y en modelos lineales desestabiliza los coeficientes.

2. Justificación de dominio:

debe responder a una hipótesis del negocio, no a una combinación aleatoria. Una variable sin interpretación semántica es difícil de validar y de mantener.

3. Validación empírica:

su inclusión debe mejorar el rendimiento del modelo medido en datos que el modelo no vio durante el entrenamiento. El riesgo de sobreajuste aumenta con cada variable adicional — más variables dan al modelo más grados de libertad para memorizar patrones espurios del train set.

1.2 Las cuatro operaciones base

Operación	Descripción	Ejemplo
Construcción de nuevas variables	Derivar columnas a partir de las existentes	<code>deuda / ingresos</code> , <code>friccion_reciente</code>

Transformaciones matemáticas	Escalar, normalizar, corregir distribuciones	$\log(\text{ingresos} + 1)$ para sesgo positivo
Codificación de categóricas	Traducir categorías a representaciones numéricas	One-Hot, embeddings, target encoding
Reducción / combinación	Comprimir o combinar variables de forma no lineal	PCA, autoencoders

2. Generación de nuevas variables con LLM

Los LLM — Large Language Models (Modelos de Lenguaje de Gran Escala) — introducen una capacidad nueva en feature engineering: integrar conocimiento externo y razonamiento simbólico en el diseño de variables, a partir de descripciones en lenguaje natural.

Feature Engineering basado en conocimiento (Knowledge-Based Feature Construction)

Históricamente dependía del criterio humano. Con LLM puede parcialmente automatizarse: el modelo recibe una descripción del problema en lenguaje natural y propone transformaciones lógicas o matemáticas relevantes, generando código funcional y reproducible (Boonstra, 2024).

2.1 El proceso en tres pasos

Flujo de generación de variables con LLM

Paso 1 — Descripción: el analista describe en lenguaje natural la lógica de la nueva variable. Ejemplo: "Si un cliente lleva más de 3 años y su gasto mensual es mayor que 100 €, se considera premium."

Paso 2 — Generación: el LLM interpreta la lógica y propone código Python, SQL u otro lenguaje que implementa la variable.

Paso 3 — Validación: el analista verifica que la lógica es correcta, útil y coherente con el objetivo del modelo. *Este paso es responsabilidad del humano y no puede omitirse.*

2.2 Tres tipos de variables que los LLM generan bien

A) Variables de interacción semántica

Combinan múltiples columnas de forma significativa sin requerir prueba exhaustiva de combinaciones. Ejemplo: a partir de `ingresos` y `educacion`, inferir una categoría socioeconómica. El LLM interpreta la lógica de segmentación y la codifica directamente.

B) Variables condicionales o basadas en lógica

Aplican reglas con múltiples condiciones anidadas para etiquetar observaciones. Ejemplo: `es_cliente_premium = 1 si antigüedad > 3 años AND gasto_mensual > 100€`. El LLM traduce instrucciones en lenguaje natural a código ejecutable.

C) Transformaciones guiadas por contexto matemático

Sugieren transformaciones estadísticas cuando se les proporciona contexto distribucional. Ejemplo: ante "la variable ingresos tiene distribución muy sesgada a la derecha", el LLM

sugiere $\log(\text{ingresos} + 1)$ para reducir asimetría y estabilizar varianza.

Ventaja diferencial frente a técnicas tradicionales

Las técnicas tradicionales operan exclusivamente sobre el contenido del dataset mediante transformaciones algebraicas o estadísticas. Los LLM pueden incorporar información externa (definiciones legales, conocimiento sectorial, comportamientos históricos) que no está en los datos, generando variables enriquecidas de alto valor predictivo. Además, su tiempo de desarrollo es bajo y su adaptabilidad a nuevos dominios es alta — a diferencia del enfoque manual que requiere capacitación específica.

3. Embeddings generativos para variables categóricas

3.1 Limitaciones del One-Hot Encoding

El One-Hot Encoding (OHE) es el enfoque clásico para codificar variables categóricas: convierte cada categoría en una columna binaria. Tiene dos limitaciones estructurales:

Limitaciones del One-Hot Encoding

Alta dimensionalidad:

con variables de alta cardinalidad (códigos postales, productos, ocupaciones con cientos de valores), el espacio de representación crece linealmente con el número de categorías, generando matrices dispersas y costosas computacionalmente.

Ausencia de semántica:

OHE trata todas las categorías como equidistantes. No distingue que "camisa" y "pantalón" son más similares entre sí que "camisa" y "móvil". Toda la información relacional entre categorías se pierde.

3.2 ¿Qué son los embeddings?

Embedding

Representación vectorial densa y de baja dimensión que ubica elementos discretos (palabras, productos, categorías) en un espacio continuo, tal que elementos semánticamente similares tienen vectores cercanos entre sí. En lugar de una codificación binaria dispersa (sparse), los embeddings capturan coocurrencias, similitud semántica y relaciones contextuales en pocas dimensiones.

Formalmente, un embedding es una función $f: \mathcal{C} \rightarrow \mathbb{R}^d$ donde $\mathcal{C}$ es el

conjunto de categorías y $d \ll |\mathcal{C}|$ es la dimensión del espacio latente. El objetivo es que:

$$\text{sim}(f(c_i), f(c_j)) \approx \text{similitud_semántica}(c_i, c_j)$$

3.3 Embeddings aprendidos desde datos: el autoencoder

Un autoencoder es una red neuronal que aprende a comprimir sus entradas en un espacio latente de baja dimensión y luego reconstruirlas. El "cuello de botella" (bottleneck) fuerza al modelo a retener solo la información esencial.

Proceso para generar embeddings con autoencoder

1. Representar la variable categórica con OHE (representación inicial dispersa).
2. Entrenar el autoencoder con arquitectura encoder → bottleneck → decoder, donde el bottleneck tiene dimensión d (por ejemplo, $d = 3$ para una categoría con 10 valores únicos).
3. El vector en la capa bottleneck para cada categoría es su embedding aprendido.

Ventajas: adaptado al dominio específico de los datos; captura semántica emergente — categorías usadas en contextos similares ocupan posiciones cercanas en el espacio latente; si se entrena con señal supervisada (incluyendo la variable objetivo), los embeddings se optimizan directamente para la tarea predictiva.

3.4 Embeddings preentrenados con LLM

Modelos como `text-embedding-ada-002` de OpenAI o `text-embedding-3-small` generan embeddings para texto libre a partir de conocimiento preentrenado en grandes corpus. Son útiles cuando las categorías son descripciones textuales o cuando el dataset propio tiene pocas muestras por categoría.

Flujo completo: de texto libre a representación predictiva

Partiendo de notas de soporte al cliente (texto libre):

Nivel 1 — Variable categórica:

LLM clasifica cada nota como POSITIVO, NEGATIVO o NEUTRAL. Variable interpretable para áreas de negocio.

Nivel 2 — Embedding denso:

modelo de embeddings genera un vector de alta dimensión (ej. 1536 dimensiones) por cada nota. Las notas semánticamente similares tienen vectores cercanos. Este vector se puede usar directamente como entrada a modelos predictivos o reducirse con PCA.

Este doble enfoque es valioso: la variable categórica facilita la comunicación con negocio; los embeddings ofrecen representación más rica para los modelos.

Autoencoder clásico vs VAE para embeddings

El autoencoder clásico aprende una representación puntual fija en el espacio latente: cada categoría mapea a un vector determinístico. El VAE — Variational Autoencoder —

aprende una distribución $\mathcal{N}(\mu, \sigma^2)$ en el espacio latente, lo que produce un

espacio más continuo y estructurado, útil para generación. Para embeddings de variables categóricas en pipelines predictivos, el autoencoder clásico es suficiente y más interpretable.

4. Selección de características e importancia de variables

Una vez ampliado el espacio de variables, el siguiente paso crítico es seleccionar cuáles retener. Esta etapa optimiza el rendimiento predictivo, mejora la interpretabilidad, reduce el riesgo de sobreajuste y disminuye la carga computacional.

Feature importance

Cuantificación de la influencia que tiene cada variable de entrada sobre la predicción de un modelo. No es intrínseca a los datos — es relativa al algoritmo y al objetivo. Permite construir modelos más precisos, eficientes y comprensibles.

4.1 Importancia basada en árboles

En cada nodo de un árbol de decisión, el algoritmo elige la variable que produce la mayor reducción de impureza (Gini o entropía). La importancia de cada variable es la suma de reducciones de impureza que produce en todos los nodos donde fue usada, promediada a lo largo de todos los árboles en un Random Forest.

Limitación: sesgo hacia variables de alta cardinalidad

Variables numéricas con muchos valores posibles (como ingresos con decimales) ofrecen más puntos de corte potenciales que variables categóricas con pocos niveles. Por pura probabilidad, es más fácil que entre tantos cortes alguno produzca una reducción de impureza alta — incluso si la variable no es realmente la más predictiva. Esto genera un sesgo sistemático hacia variables numéricas de alta cardinalidad en el ranking de importancia.

4.2 Importancia por permutación (Permutation Importance)

Permutation Importance — procedimiento

1. Entrenar y evaluar el modelo en el conjunto de test. Registrar la métrica base (ej. F1 = 0.70).

2. Tomar la columna j y desordenar sus valores aleatoriamente entre registros.

Esto rompe la relación entre esa variable y el target.

3. Evaluar el modelo con la columna j permutada. Registrar la nueva métrica.

4. La importancia de la variable j es la caída en la métrica:

$$\text{imp}_j = \text{métrica}_{\text{original}} - \text{métrica}_{\text{permutada}} \quad .$$

5. Repetir múltiples veces para estabilidad estadística y promediar.

Ventaja clave: es *agnóstico al modelo* (model-agnostic) — funciona con cualquier algoritmo, no solo con árboles. No depende de la estructura interna del modelo sino únicamente de su comportamiento en predicción.

Desventaja: computacionalmente costoso, requiere reevaluar el modelo una vez por variable por repetición.

4.3 SHAP — SHapley Additive exPlanations

SHAP (Lundberg y Lee, 2017)

Framework de interpretabilidad basado en los valores de Shapley de teoría de juegos. Cuantifica la contribución de cada variable a la predicción de una observación específica, promediando su contribución marginal sobre todas las posibles combinaciones de las demás variables.

La intuición: las variables son jugadores de un equipo y la predicción es el resultado del partido. Para repartir el crédito justamente, se calcula la contribución marginal de cada jugador en todas las coaliciones posibles.

Formalmente, el valor SHAP de la variable j para la observación i es:

$$\phi_j(i) = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [f(S \cup \{j\}) - f(S)]$$

donde F es el conjunto de todas las variables, S es un subconjunto de variables

que no incluye j , y $f(S)$ es la predicción del modelo usando solo las variables

en S .

Propiedad de eficiencia (garantía matemática)

Los valores SHAP de todas las variables suman exactamente la diferencia entre la predicción individual y la predicción promedio del modelo:

$$\sum_{j=1}^{|F|} \phi_j(i) = f(x_i) - \mathbb{E}[f(x)]$$

Esto no es una aproximación — es una garantía derivada de los axiomas de Shapley.

4.4 Interpretabilidad local vs global

Por qué SHAP importa a nivel de registro individual

Importancia por permutación dice: "en promedio, `friccion_reciente` es la variable más importante del modelo." Pero para el cliente C-00047 específicamente, los valores SHAP podrían revelar:

- `friccion_reciente` : +0.25 (empuja hacia churn)
- `llamadas_soporte` : +0.12
- `antiguedad_meses` : +0.08
- `consumo_gb` : +0.02

Suma: 0.47 = predicción del cliente (0.82) – predicción promedio del modelo (0.35).

Para el cliente C-00048, el ranking podría ser completamente distinto. Esa granularidad permite personalizar intervenciones de negocio — el equipo de retención sabe exactamente qué factor accionar para cada cliente.

La importancia global con SHAP se obtiene promediando el valor absoluto de los SHAP values de cada variable a través de todos los registros:

$$\text{importanciaglobal}_j = \frac{1}{n} \sum_{i=1}^n |\phi_j(i)| \quad . \text{ Esto preserva la información local sin}$$

perderla.

4.5 Tres roles del LLM en la selección de variables

Rol	Descripción	Qué aporta que los métodos estadísticos no pueden
Generador de código	Genera pipelines completos para aplicar SHAP, permutation importance, Lasso, RFE — adaptados al contexto del dataset	Velocidad y adaptabilidad al contexto sin necesidad de que el analista escriba el código desde cero

Experto de dominio	Propone qué variables incluir, fusionar o descartar según lógica de negocio, interpretabilidad esperada o restricciones organizacionales	Conocimiento semántico del dominio. Un método estadístico puede decir "descartá plan_contratado_Premium" — el LLM puede argumentar por qué conservarla tiene sentido estratégico
Intérprete de resultados	Convierte tablas de importancia de variables en narrativas ejecutivas comprensibles para perfiles no técnicos	Traducción de números en acciones de negocio accionables

El LLM como "curador semántico" (Boonstra, 2024)

El papel del LLM no es el de un filtro estadístico, sino el de un curador que guía la construcción del modelo hacia estructuras más alineadas con el conocimiento experto y las metas organizacionales. Complementa los métodos estadísticos, no los reemplaza.

5. Evaluación del impacto: el concepto de uplift

Uplift

Diferencia en la métrica de evaluación entre el modelo enriquecido (con variables nuevas) y el modelo baseline (con variables originales), manteniendo constantes el algoritmo, los hiperparámetros y el split de datos:

$$Uplift = Métrica_{enriquecido} - Métrica_{baseline}$$

Ejemplo: si $F1_{baseline} = 0.62$ y $F1_{enriquecido} = 0.70$,

entonces $Uplift = +0.08$.

5.1 Protocolo de evaluación formal

Tres pasos del protocolo de evaluación

Paso 1 — Baseline: entrenar el modelo con las variables originales. Evaluar en conjunto de test (hold-out o validación cruzada). Registrar métricas: accuracy, F1, AUC para clasificación; RMSE para regresión.

Paso 2 — Modelo enriquecido: entrenar el *mismo* modelo con los *mismos* hiperparámetros, incorporando las variables nuevas. Todo lo demás constante — esa es la condición para que la comparación sea válida.

Paso 3 — Medición: calcular el uplift. El material referencia como umbrales orientativos +0.07 en AUC o +0.10 en F1. El uplift debe ser consistente en múltiples segmentos de validación para descartar que sea azar.

Uplift global pequeño puede esconder valor de negocio alto

Un uplift de +0.02 en F1 global puede ser irrelevante. Pero si ese uplift se concentra en el segmento de clientes de alto valor, el impacto económico puede ser enorme. SHAP permite detectar este patrón: filtrando los SHAP values solo para ese subgrupo, se puede ver si las variables nuevas contribuyen mucho más ahí que en el promedio general.

5.2 Contribución del LLM en la evaluación

El LLM contribuye en dos niveles en esta fase. Primero, en la **sugerencia de métricas**: dado el tipo de problema (clasificación binaria desbalanceada, multiclase, regresión), el LLM recomienda la métrica apropiada — evitando, por ejemplo, usar accuracy en problemas con clases muy desbalanceadas donde es engañosa. Segundo, en la **interpretación automática**: convierte la tabla comparativa de métricas en un informe narrativo accionable para audiencias no técnicas.

6. Feature engineering en pipelines de ML

En entornos de producción, la ingeniería de características no puede limitarse a scripts aislados. Debe integrarse en un **pipeline estructurado y reproducible** que encapsule todas las transformaciones desde la entrada cruda hasta la predicción.

Pipeline de ML como DAG — Directed Acyclic Graph (Grafo Acíclico Dirigido)

Estructura donde cada nodo representa una operación de transformación (imputar, escalar, codificar, generar variable nueva) y cada arista representa una dependencia entre operaciones. "Acíclico" garantiza que los datos fluyen en una sola dirección — no hay circularidad. Implementado en scikit-learn con `Pipeline` y `ColumnTransformer`.

6.1 Tres problemas críticos que el pipeline resuelve

Data leakage (fuga de datos)

Si se calcula la media de `consumo_gb` para imputar valores faltantes usando el dataset completo (train + test), se filtra información del futuro al modelo durante el entrenamiento. Dentro de un pipeline, cada transformación se ajusta (`fit`) exclusivamente con el train set y luego se aplica (`transform`) al test set con los parámetros aprendidos del train. Esto previene la fuga.

Training-serving skew

Ocurre cuando las transformaciones aplicadas durante el entrenamiento no se replican de forma idéntica en producción (serving). Ejemplo: el `StandardScaler` usa la media y desviación del train set. Si en producción se recalculan con datos nuevos o con una fórmula diferente, el modelo recibe valores en una escala distinta a la que aprendió — y falla silenciosamente, sin error explícito. El pipeline encapsula el scaler entrenado y lo aplica automáticamente en predicción.

Reproducibilidad

Todas las transformaciones quedan registradas en el pipeline como un objeto serializable. Cualquier reentrenamiento futuro aplica exactamente las mismas reglas, sin depender de la memoria de quien ejecutó el notebook original.

6.2 Integración de LLM y embeddings como nodos del pipeline

Tres niveles de integración en el pipeline del Tema 5

Nodo 1 — LLM features:

`FunctionTransformer` que llama al LLM para generar 2 variables nuevas. Se ejecuta en cada reentrenamiento con el mismo prompt, garantizando reproducibilidad.

Nodo 2 — Embeddings SVD:

`CategoricalSVDEmbedding` (estimador personalizado con `fit/transform`) que aprende embeddings densos para `plan_contratado` a partir de coocurrencias con variables numéricas discretizadas. El `fit` se aplica solo sobre `train` — sin fuga.

Nodo 3 — Preprocesamiento estándar:

`ColumnTransformer` con `StandardScaler` para numéricas y `OneHotEncoder` para categóricas. Selectores dinámicos (`make_column_selector`) para adaptarse a las columnas nuevas generadas por el LLM.

Agentes LLM en pipelines adaptativos

El nivel más avanzado de integración: agentes LLM (LangChain, CrewAI) como nodos del pipeline que se activan automáticamente cuando el rendimiento cae por debajo de un umbral. El agente analiza el dataset actual, propone nuevas variables derivadas, y las incorpora al ciclo siguiente. Esto representa el paso de pipelines estáticos a *flujos de aprendizaje adaptativos y aumentados por IA generativa*.

7. Comparación: técnicas tradicionales vs IA generativa

Tarea	Método tradicional	Con IA generativa	Ventaja clave
Crear interacciones	Combinaciones manuales (ej. edad × ingresos)	Prompt: "Sugiere 3 interacciones que podrían ser predictivas"	Descubre relaciones no obvias; incorpora conocimiento de dominio implícito
		Embeddings aprendidos con	

Codificar categóricas	One-Hot Encoding, Label Encoding	autoencoders o generados por LLM	Captura semántica, reduce dimensionalidad, mejora generalización
Transformar texto libre	Bag-of-Words, TF-IDF	Extracción de entidades, sentimiento, clasificación contextual con LLM	Traduce lenguaje natural en variables predictivas con alto poder explicativo
Evaluar importancia	Árboles, permutaciones, SHAP, Gain	Prompt sobre relevancia según lógica del negocio + interpretación con SHAP	Asistencia en priorización y generación de reportes ejecutivos

Responsabilidad del analista humano — irremplazable

Intencionalidad:

definir qué problema se resuelve, qué hipótesis de negocio guían la creación de variables, qué restricciones existen (costo, interpretabilidad, regulación). El LLM puede sugerir 20 variables — el analista decide cuáles tienen sentido.

Validación:

el LLM puede generar alucinaciones y reglas incorrectas si es mal instruido. El analista es el filtro final. Esta responsabilidad no puede delegarse. Es coherente con la visión del *analista aumentado* (augmented analyst): la IA amplía la capacidad exploratoria del experto, no lo reemplaza.

8. Síntesis: cadena conceptual del Tema 5

Flujo completo integrado

Datos crudos → identificar hipótesis de negocio → **LLM genera variables** (interacción semántica, condicionales, transformaciones) → validación humana → **embeddings** para categóricas de alta cardinalidad (autoencoder o LLM preentrenado) → **selección de características** (importancia por árboles + permutación + SHAP) con LLM como curador semántico → **evaluación de uplift** (baseline vs enriquecido, global y por subgrupos) → **pipeline DAG** reproducible con nodos LLM y embeddings → **producción** libre de data leakage y training-serving skew.

9. Preguntas de autoevaluación

Preguntas para verificar comprensión

1.

¿Por qué la importancia basada en árboles tiene sesgo hacia variables numéricas de alta cardinalidad? ¿Cómo lo corrige la importancia por permutación?

2.

SHAP garantiza que los valores de todas las variables sumen la diferencia entre la predicción individual y la predicción promedio. ¿Qué nombre recibe esta propiedad y por qué es importante para la interpretabilidad?

3.

Describí un escenario donde el uplift global es +0.02 en F1 pero el valor de negocio es muy alto. ¿Qué herramienta usarías para detectar ese patrón?

4.

¿Cuál es la diferencia entre data leakage y training-serving skew? ¿Cómo el pipeline de scikit-learn previene ambos?

5.

Un embedding aprendido con autoencoder y un embedding generado por `text-embedding-ada-002` de OpenAI: ¿en qué contexto usarías cada uno? ¿Cuál captura mejor el dominio específico de tus datos?

6.

El LLM puede actuar como generador de código, experto de dominio e intérprete de resultados en la selección de variables. ¿Cuál de los tres roles aporta algo que los métodos estadísticos puros no pueden dar? Justificá.

7.

¿Por qué el sobreajuste aumenta al agregar más variables y cómo se detecta ese problema en la práctica?

10. Referencias

- Boonstra, L. (2024). *Prompt Engineering*. Google.
- Chollet, F. (2021). *Deep Learning with Python*. Simon y Schuster.
- Domingos, P. (2012). A few useful things to know about machine learning. *Communications of the ACM*, 55(10), 78–87. <https://doi.org/10.1145/2347736.2347755>
- Hollmann, N., Müller, S. y Hutter, F. (2023). Large language models for automated data science: Introducing CAAFE for context-aware automated feature engineering. *NeurIPS 2023*, 36, 44753–44775.
- Ko, J., Park, G., Lee, D. y Lee, K. (2025). FeRG-LLM: Feature engineering by reason generation large language models. *ArXiv* 2503.23371.
- Kuhn, M. y Johnson, K. (2019). *Feature Engineering and Selection: A Practical Approach for Predictive Models*. Chapman and Hall/CRC. <https://doi.org/10.1201/9781315108230>
- Lundberg, S. M. y Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 4765–4774.